

Decision Trees and Ensemble Methods

EECS 492

Introduction to Artificial
Intelligence

Reminders

- This week's discussion on decision trees and linear regression
- HW 1 to be released by tomorrow
 - Start early!
 - Topics: Agents and ML
 - Coding resources available in Canvas
 - Files > Coding Resources
 - Includes video guide on virtual environments
- HW1 due date: 10/6

Today's Class

- **Decision Trees**
- **Ensemble Approaches**

An Example Classification Problem

- Target: Will we or won't we wait for a table at a restaurant
- Attributes:
 - Alternate – do we have another option?
 - Bar – is there a bar area we can wait in?
 - Fri/Sat – is it Friday or Saturday (true/false)?
 - Hungry – are we?
 - Patrons – how many people (none, some, full)?
 - Price – how pricy (\$, \$\$, \$\$\$)?
 - Raining – is it?
 - Reservation – did we make one?
 - Type – food (French, Italian, Thai, burger)?
 - WaitEstimate – how long to wait (0-10, 10-30, 30-60, >60 min)?

The Restaurant Table

Attributes also known as features
(notation: x)

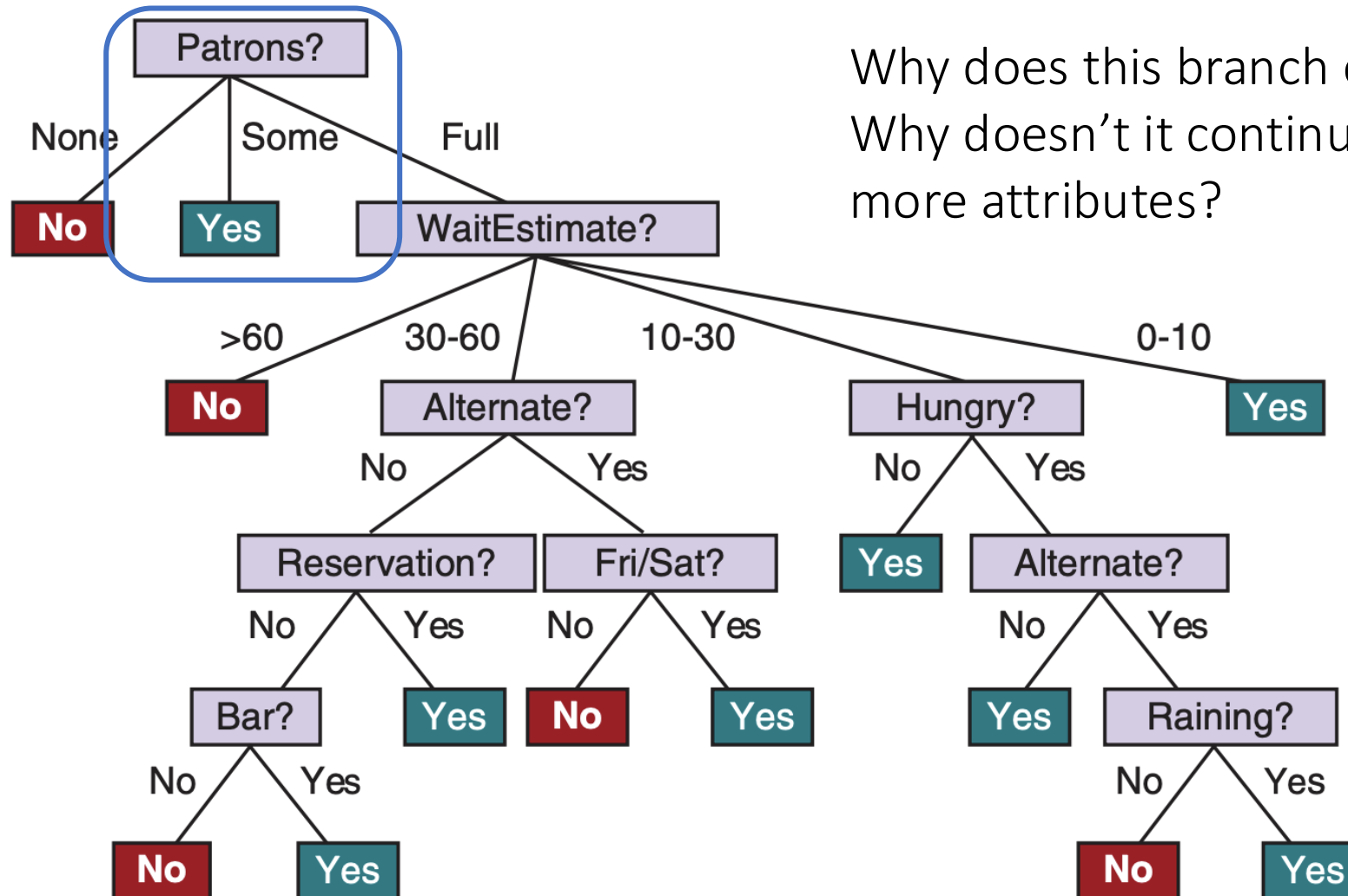
Target or label
(notation: y)

Examples,
observations,
or data points

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
x_1	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0-10</i>	$y_1 = \text{Yes}$
x_2	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30-60</i>	$y_2 = \text{No}$
x_3	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0-10</i>	$y_3 = \text{Yes}$
x_4	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10-30</i>	$y_4 = \text{Yes}$
x_5	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	$y_5 = \text{No}$
x_6	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0-10</i>	$y_6 = \text{Yes}$
x_7	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0-10</i>	$y_7 = \text{No}$
x_8	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0-10</i>	$y_8 = \text{Yes}$
x_9	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	$y_9 = \text{No}$
x_{10}	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10-30</i>	$y_{10} = \text{No}$
x_{11}	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0-10</i>	$y_{11} = \text{No}$
x_{12}	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30-60</i>	$y_{12} = \text{Yes}$



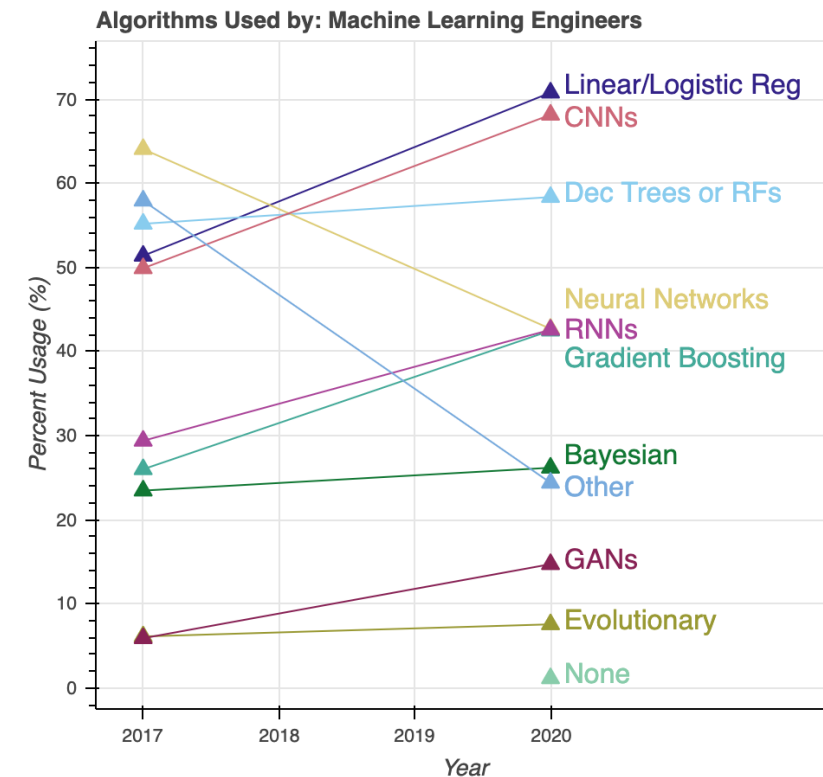
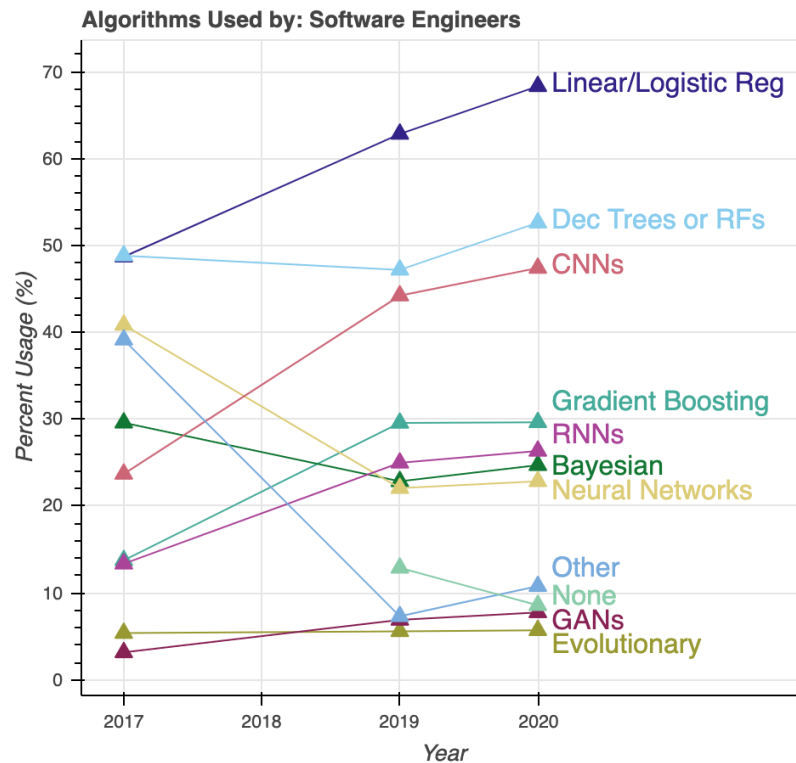
A Decision Tree for the Restaurant Example



Why does this branch end at “Yes”?
Why doesn't it continue to branch on more attributes?

Decision Tree Classification

- Simple supervised learning model for classification



Source: Kaggle State of Data Science
and Machine Learning 2017-2020

Decision Trees and Ensemble Methods

Decision Tree Classification

- Simple supervised learning model for classification
- Structure:
 - Input: vector of attribute values (A_i)
 - Output: decision
- How is decision made?
 - Sequence of tests
 - **Internal nodes** – test of attribute values
 - **Branch** – labeled with possible values of attribute
 - **Leaf** – value to be returned by function

Simplification: Boolean Classification!

Inducing DTs from Examples

- Let's consider the restaurant example again
 - $n = 10$ variables
 - How do we find the smallest tree possible (that is consistent with the data)?
- Intractable problem: 2^{2^n} trees!
 - What if we employ some heuristics?
 - We can find an approximate solution!
- This gives us motivation for the **DECISION-TREE-LEARNING** algorithm!

Overview of DECISION-TREE-LEARNING

- Greedy divide-and-conquer algorithm
 - Always test the most important attribute first!
- What does “most important” mean?
 - General: the one that has the biggest impact on the classification of an example
- Goal – correct classification with smallest number of tests
 - Short paths
 - Shallow trees!



DECISION-TREE-LEARNING

function LEARN-DECISION-TREE (*examples*, *attributes*, *parent_examples*) **returns** a tree

if *examples* is empty **then return** PLURALITY-VALUE(*parent_examples*)
else if all *examples* have the same classification **then return** classification
else if *attributes* is empty **then return** PLURALITY-VALUE(*examples*)
else

$A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$

tree $\leftarrow$ a new decision tree with root test *A*

for each value *v* of *A* **do**

$\text{exs} \leftarrow \{e: e \in \text{examples} \text{ and } e.A = v\}$

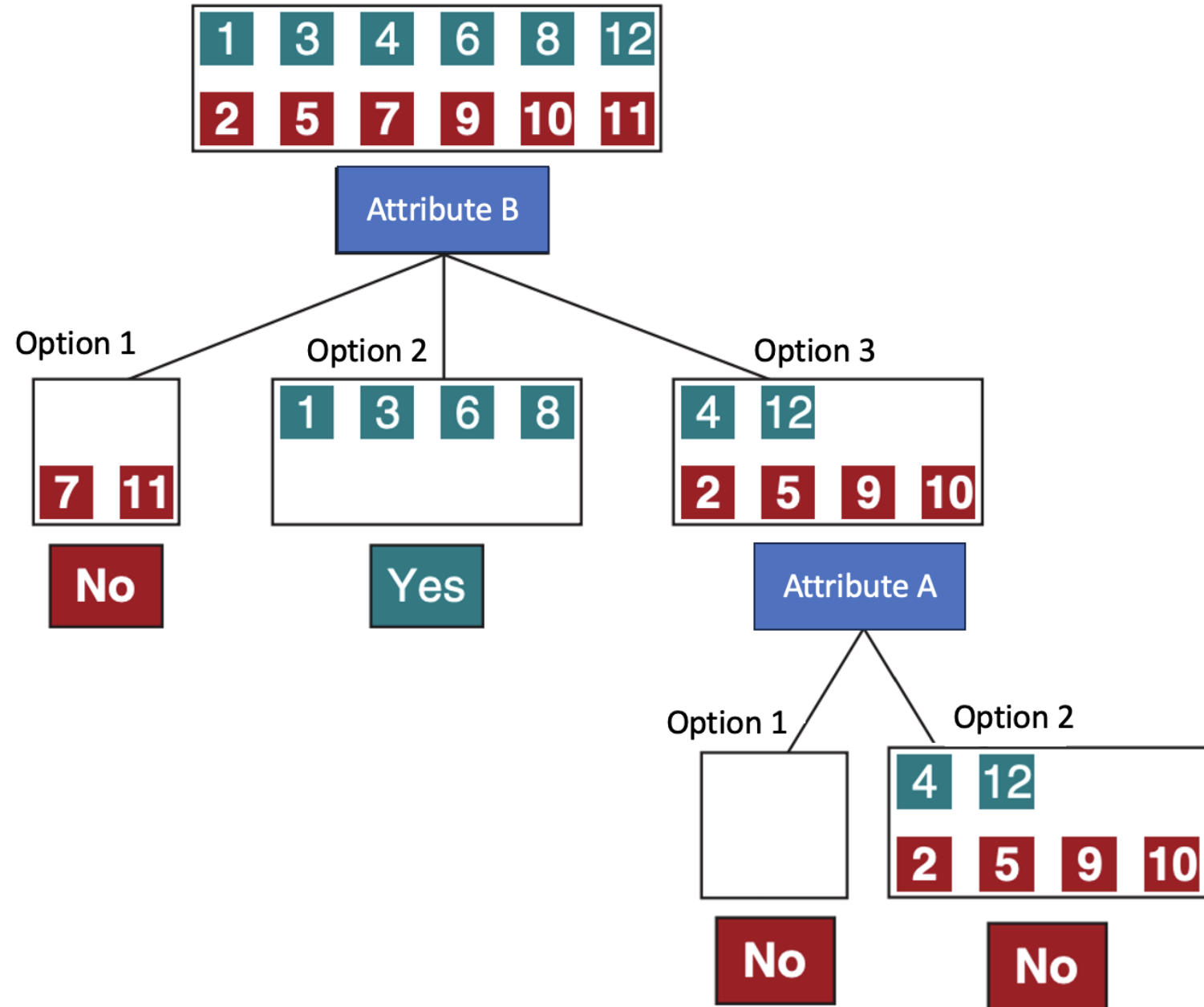
subtree $\leftarrow$ LEARN-DECISION-TREE(*exs*, *attributes*−*A*, *examples*)

add a branch to *tree* with label (*A* = *v*) and subtree *subtree*

return *tree*

Example DT

- Positive examples – blue
- Negative examples - red



Sketch of DECISION-TREE-LEARNING

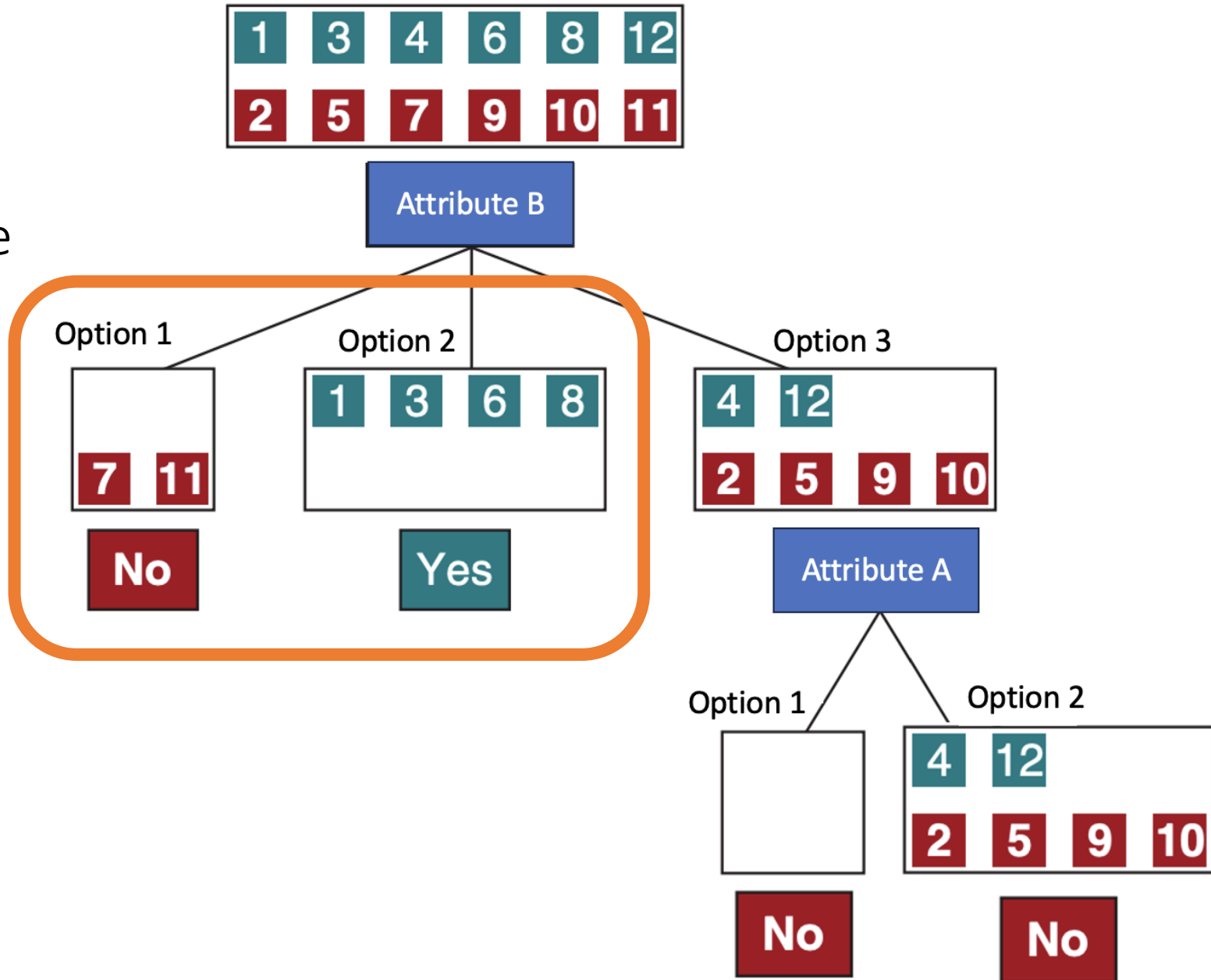
- First attribute test splits up examples
 - Each outcome is a new DT learning problem!
- Four cases:
 1. Are all remaining cases positive (or negative)? – Done!
 2. Are there some positive and some negative? – Find the best attribute and split!
 3. No examples left? No data supports combination of attribute values – return default value from plurality
 4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
 - Each outcome is a new DT learning problem!
- Four cases:
 1. Are all remaining cases positive (or negative)? – Done!
 2. Are there some positive and some negative? – Find the best attribute and split!
 3. No examples left? No data supports combination of attribute values – return default value from plurality of parent
 4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Example DT

- Positive examples – blue
- Negative examples - red

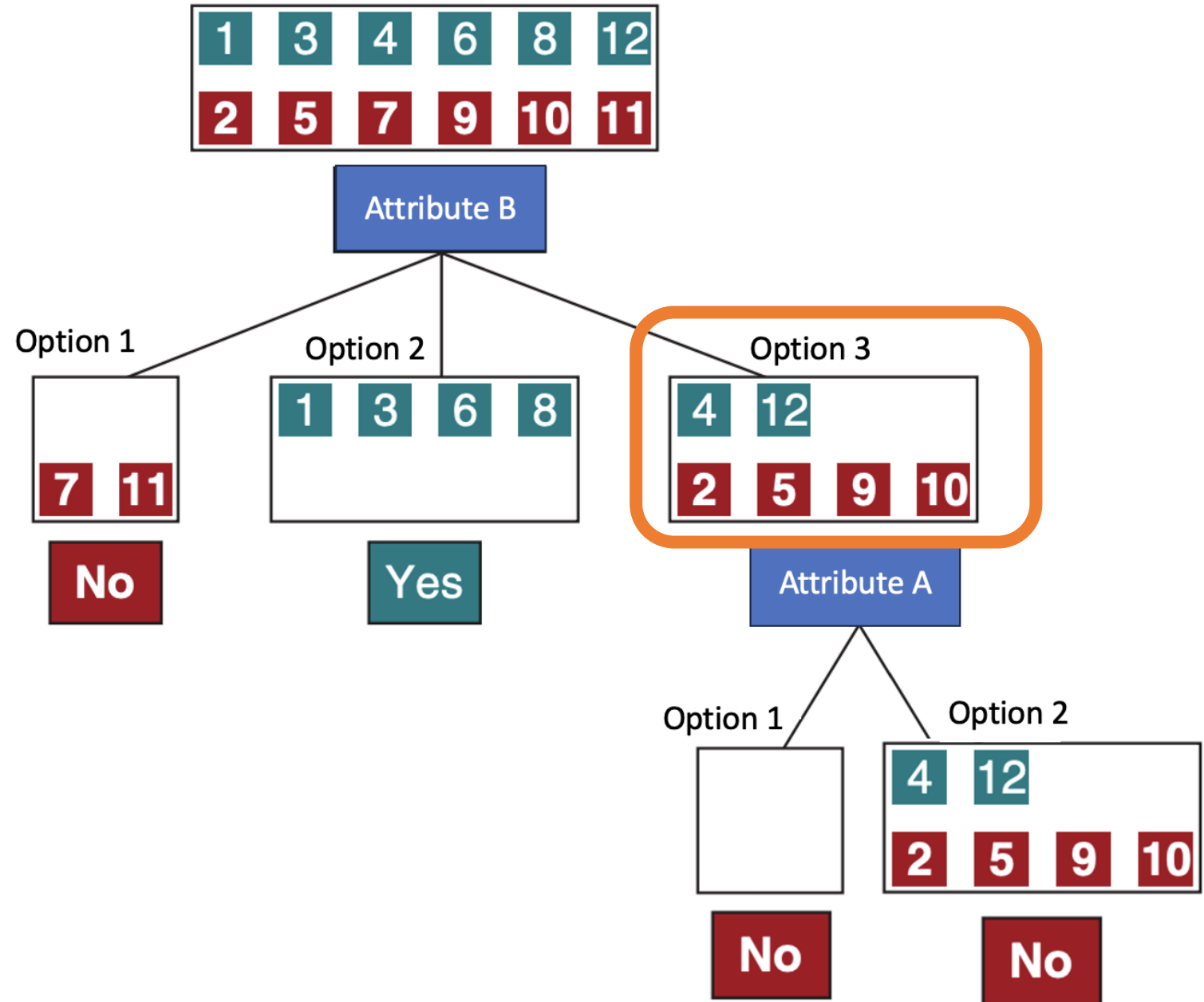


Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
 - Each outcome is a new DT learning problem!
- Four cases:
 1. Are all remaining cases positive (or negative)? – Done!
 2. Are there some positive and some negative? – Find the best attribute and split!
 3. No examples left? No data supports combination of attribute values – return default value from plurality of parent
 4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Example DT

- Positive examples – blue
- Negative examples - red

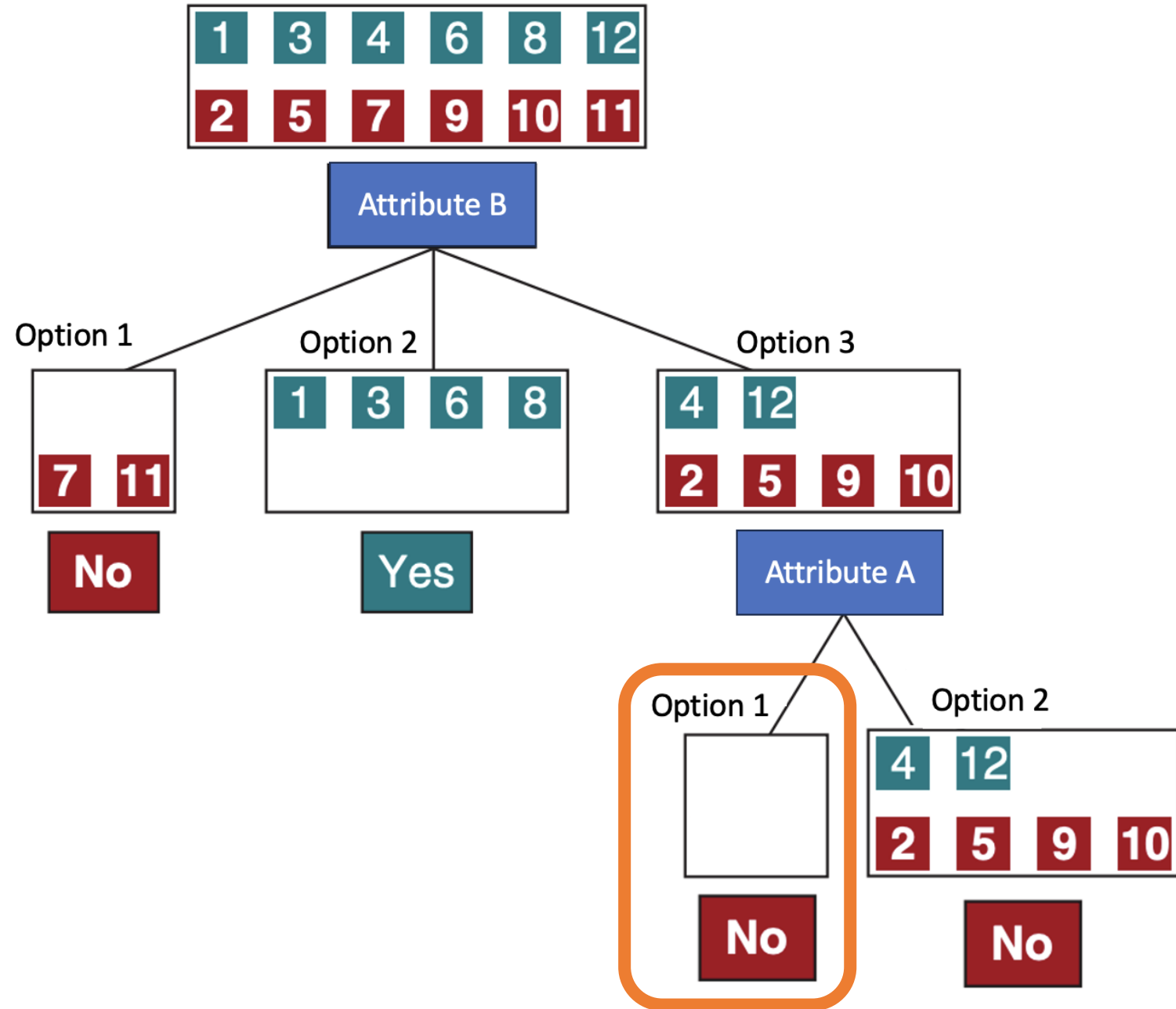


Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
 - Each outcome is a new DT learning problem!
- Four cases:
 1. Are all remaining cases positive (or negative)? – Done!
 2. Are there some positive and some negative? – Find the best attribute and split!
 3. No examples left? No data supports combination of attribute values – return default value from plurality of parent
 4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Example DT

- Positive examples – blue
- Negative examples - red

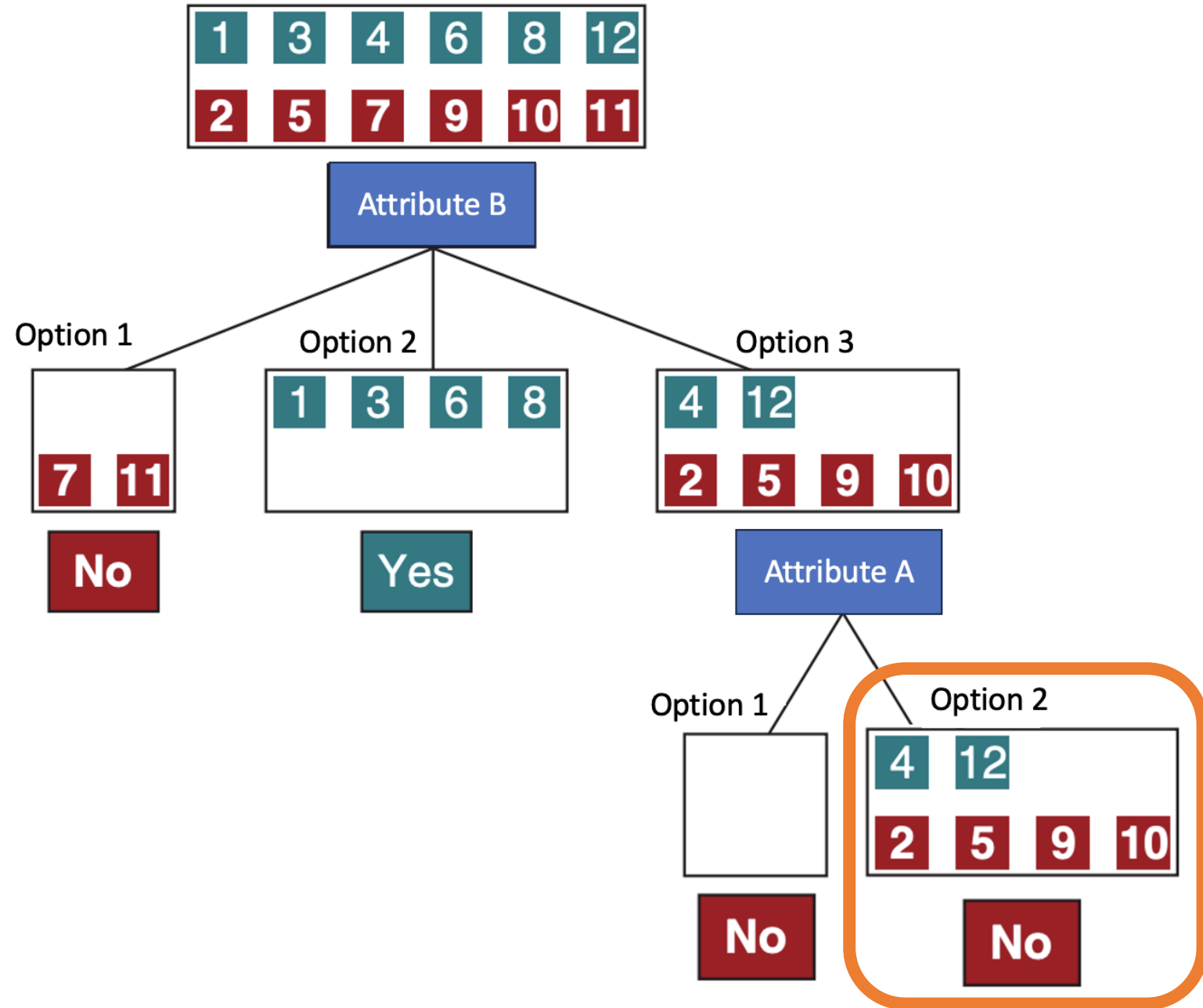


Sketch of DECISION-TREE-LEARNING

- First attribute test splits up examples
 - Each outcome is a new DT learning problem!
- Four cases:
 1. Are all remaining cases positive (or negative)? – Done!
 2. Are there some positive and some negative? – Find the best attribute and split!
 3. No examples left? No data supports combination of attribute values – return default value from plurality of parent
 4. No attributes left but still positive and negative examples? – Error or noise in the data – return plurality

Example DT

- Positive examples – blue
- Negative examples - red

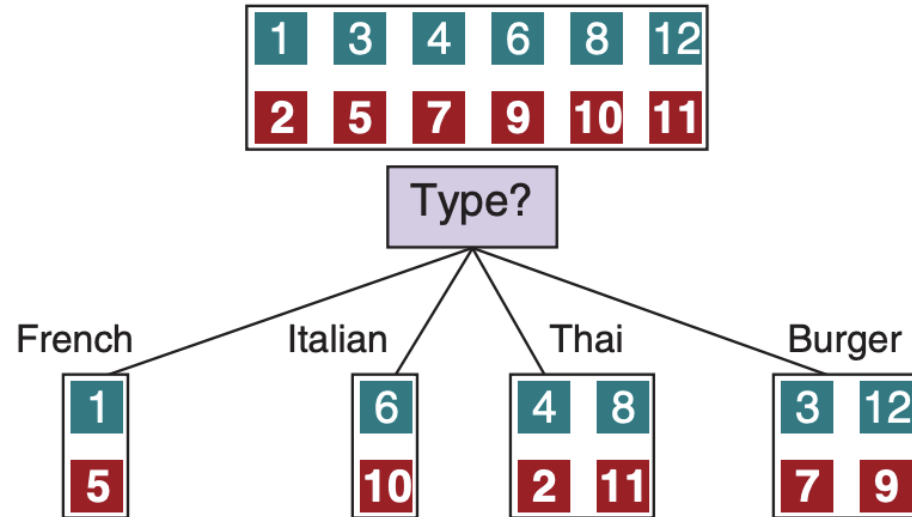


DT Points to Consider

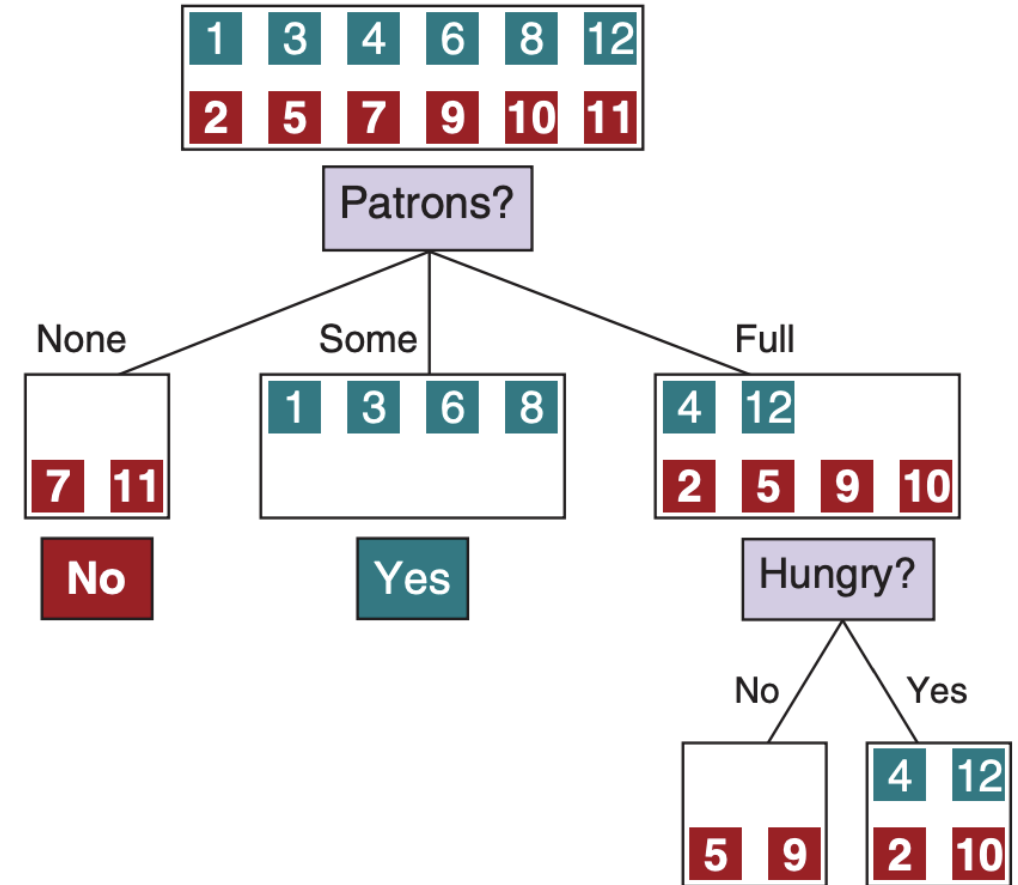
- Important notes:
 - Examples are crucial to tree construction
 - Examples do not appear in the tree itself
 - Tree: just tests on attribute values
- When might the DT make mistakes
 - Errors in the data
 - Noise in the data
 - Cases where it has seen no examples



Which tree is better?



(a)



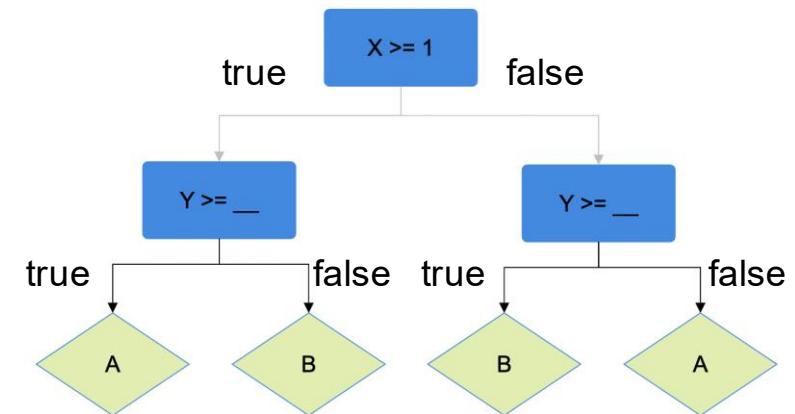
(b)

Decision Tree

Suppose we want to create a decision tree to completely classify the following dataset.

Fill in the missing values to create a consistent decision tree with the dataset.

X - Value	Y - Value	Classification
2	2	A
-3	-4	A
0	3	B
4	-1	A
3	-4	B
-2	1	B
-1	-1	A

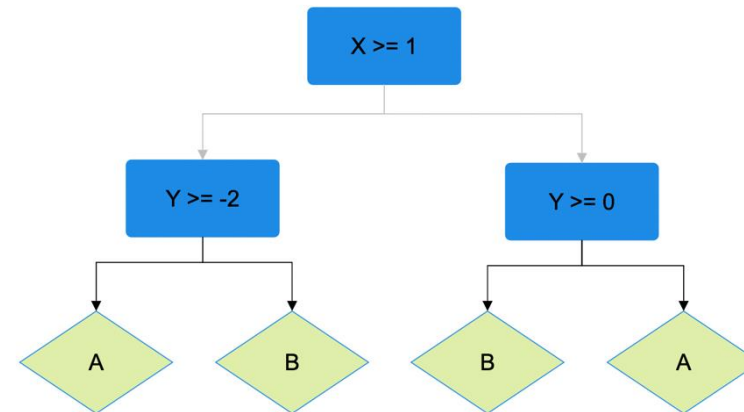


Decision Tree

Suppose we want to create a decision tree to completely classify the following dataset.

Fill in the missing values to create a consistent decision tree with the dataset. (Left in the decision tree is True, right is False)

X - Value	Y - Value	Classification
2	2	A
-3	-4	A
0	3	B
4	-1	A
3	-4	B
-2	1	B
-1	-1	A



DECISION-TREE-LEARNING

function LEARN-DECISION-TREE (*examples*, *attributes*, *parent_examples*) **returns** a tree

if *examples* is empty **then return** PLURALITY-VALUE(*parent_examples*)
else if all *examples* have the same classification **then return** classification
else if *attributes* is empty **then return** PLURALITY-VALUE(*examples*)
else

$A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$

tree $\leftarrow$ a new decision tree with root test *A*

for each value *v* of *A* **do**

$\text{exs} \leftarrow \{e: e \in \text{examples} \text{ and } e.A = v\}$

subtree $\leftarrow$ LEARN-DECISION-TREE(*exs*, *attributes*−*A*, *examples*)

add a branch to *tree* with label (*A* = *v*) and subtree *subtree*

return *tree*

How to Choose Attribute Tests

- Our algorithm hinges on finding the *most important* attributes to split on – how can we measure this?
- We will use a measure called **information gain**
 - Split on the attribute with the highest information gain

How can we calculate information gain?

- Information gain is defined in terms of entropy
- **Entropy** (H) = a measure of the uncertainty of a random variable
 - More information, less entropy
- Examples
 - A coin that always comes up heads $\rightarrow$ 0 bits
 - A fair coin (equally likely to be heads or tails) $\rightarrow$ 1 bits
 - Fair four-sided die $\rightarrow$ 2 bits

Entropy

- We define entropy as:

$$H(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)} = - \sum_k P(v_k) \log_2 P(v_k)$$

- V is a random variable with values v_k having probability $P(v_k)$
- Example: entropy of a fair coin



Exercise: Entropy

- Calculate the entropy of a loaded coin where $P(heads) = 0.99$
- Hw: Calculate the entropy of a four-sided die

Entropy for Boolean Random Variables

- Let's define $B(q)$: the entropy of a Boolean random variable that is true with probability q

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q))$$

- The loaded coin (from the previous slide) is a Boolean random variable, $q = 0.99$

- $H(\text{Loaded}) = B(q) \approx 0.08$

Back to decision trees...

- We are trying to find a way to choose which attribute to split on
 - Split on the attribute with the highest information gain
- Information gain of attribute A = the expected reduction of entropy after splitting on A

$$Gain(A) = H(Goal) - Remainder(A)$$

The entropy of the output variable
(before splitting on A)



The expected entropy remaining
after splitting on A



Calculating Gain

$$\text{Gain}(A) = H(\text{Goal}) - \text{Remainder}(A)$$

The entropy of the output variable
(before splitting on A)

The expected entropy remaining
after splitting on A

- How can we calculate $H(\text{Goal})$?
 - This is the entropy of the goal attribute
 - Let's say we have p positive examples and n negative examples

$$H(\text{Goal}) = B\left(\frac{p}{p+n}\right)$$

$$H(\text{Goal}) = -\left(\frac{p}{p+n} \log_2 \frac{p}{p+n} + \left(1 - \frac{p}{p+n}\right) \log_2 \left(1 - \frac{p}{p+n}\right)\right)$$

Probability of the positive class

Probability of the negative class

Exercise: What is $H(Goal)$? $H(Goal) = B\left(\frac{p}{p+n}\right)$

Example	Input Attributes										Goal
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>	<i>WillWait</i>
$\mathbf{x}_1$	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>0–10</i>	$y_1 = \text{Yes}$
$\mathbf{x}_2$	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>30–60</i>	$y_2 = \text{No}$
$\mathbf{x}_3$	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>Some</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	$y_3 = \text{Yes}$
$\mathbf{x}_4$	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Thai</i>	<i>10–30</i>	$y_4 = \text{Yes}$
$\mathbf{x}_5$	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>French</i>	<i>>60</i>	$y_5 = \text{No}$
$\mathbf{x}_6$	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Italian</i>	<i>0–10</i>	$y_6 = \text{Yes}$
$\mathbf{x}_7$	<i>No</i>	<i>Yes</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>0–10</i>	$y_7 = \text{No}$
$\mathbf{x}_8$	<i>No</i>	<i>No</i>	<i>No</i>	<i>Yes</i>	<i>Some</i>	<i>\$\$</i>	<i>Yes</i>	<i>Yes</i>	<i>Thai</i>	<i>0–10</i>	$y_8 = \text{Yes}$
$\mathbf{x}_9$	<i>No</i>	<i>Yes</i>	<i>Yes</i>	<i>No</i>	<i>Full</i>	<i>\$</i>	<i>Yes</i>	<i>No</i>	<i>Burger</i>	<i>>60</i>	$y_9 = \text{No}$
$\mathbf{x}_{10}$	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$\$\$</i>	<i>No</i>	<i>Yes</i>	<i>Italian</i>	<i>10–30</i>	$y_{10} = \text{No}$
$\mathbf{x}_{11}$	<i>No</i>	<i>No</i>	<i>No</i>	<i>No</i>	<i>None</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Thai</i>	<i>0–10</i>	$y_{11} = \text{No}$
$\mathbf{x}_{12}$	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Yes</i>	<i>Full</i>	<i>\$</i>	<i>No</i>	<i>No</i>	<i>Burger</i>	<i>30–60</i>	$y_{12} = \text{Yes}$

Calculating Gain

$$\text{Gain}(A) = H(\text{Goal}) - \text{Remainder}(A)$$

The entropy of the output variable
(before splitting on A)

The expected entropy remaining
after splitting on A

- How can we calculate *Remainder(A)*?
 - Let's say attribute A has d distinct values
 - Divides training set into subsets $E_1, \dots, E_d$
 - Each subset E_k has p_k positive, n_k negative examples
 - If we split the decision tree on A , and go along the branch with subset E_k , how many bits of information will we need to answer the question?

$$B\left(\frac{p_k}{p_k + n_k}\right)$$

- Summing over all values k , we can take the *expected entropy*:

$$\text{Remainder}(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right)$$

How much randomness
is in the examples with
that value?

How likely is the k^{th} value?

$$\text{Gain}(A) = H(\text{Goal}) - \text{Remainder}(A)$$

Exercise: Gain

The entropy of the output variable
(before splitting on A)



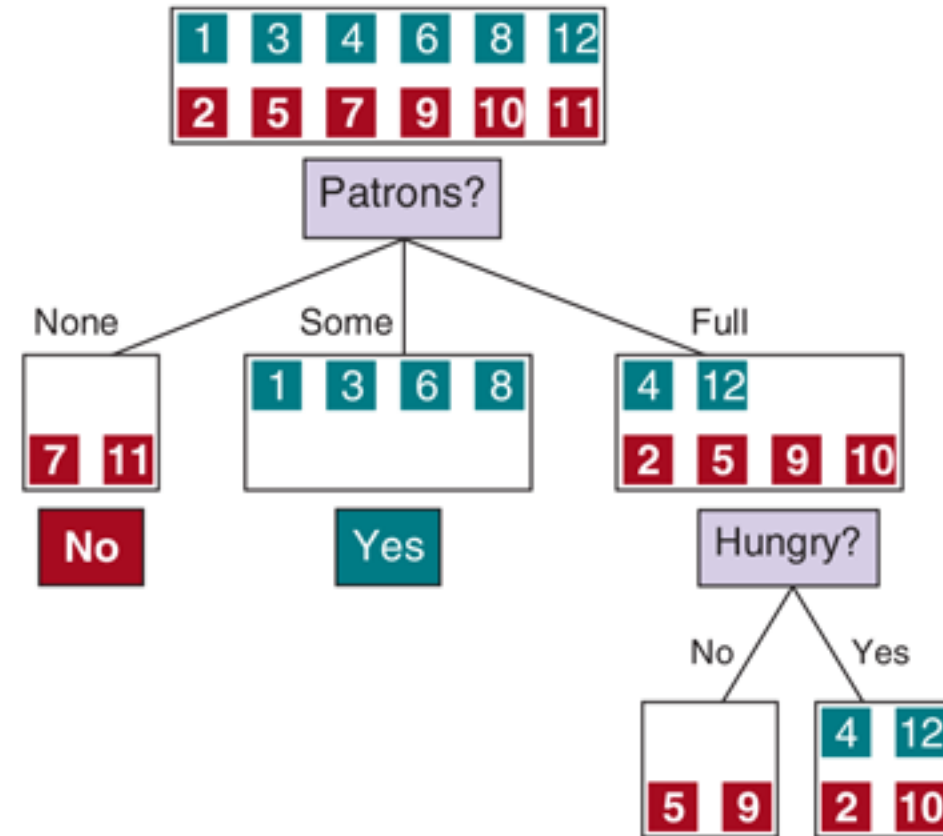
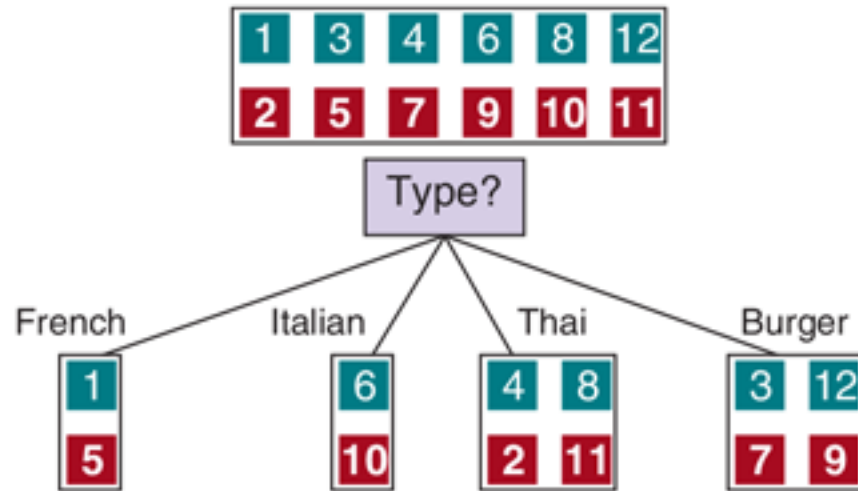
The expected entropy remaining
after splitting on A



- What is the information gain of Patrons?
- What is the information gain of Type?
- If you were just choosing between these two attributes, which attribute would you split on?



Let's Analyze Two of the Possible Splits



(b)

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q))$$

$$Remainder(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right)$$

$$Gain(A) = B\left(\frac{p}{p + n}\right) - Remainder(A)$$

$$B(1) = 0; B(0) = 0; B(1/2) = 1$$

$$B(1/3) = B(2/3) \approx 0.92; B(1/4) = B(3/4) \approx 0.81$$

Problems with DT

- Are trees always appropriate?
 - Think about rolling a die – does a tree make sense?
- What is the danger?
 - **Overfitting** – creating a model that closely matches your training data without knowing if it can generalize!
- Possible solution
 - Decision tree pruning
 - Remove tests that have only noise as their outcome [for more detail, check out chapter 19.3.4]
 - Have many small trees and combine them → **Ensemble methods!**

Decision Trees

- Simple model for classification

Example	Input Attributes									
	<i>Alt</i>	<i>Bar</i>	<i>Fri</i>	<i>Hun</i>	<i>Pat</i>	<i>Price</i>	<i>Rain</i>	<i>Res</i>	<i>Type</i>	<i>Est</i>
x_1	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10
x_2	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60
x_3	No	Yes	No	No	Some	\$	No	No	Burger	0-10
x_4	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30
x_5	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60
x_6	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10
x_7	No	Yes	No	No	None	\$	Yes	No	Burger	0-10
x_8	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10
x_9	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60
x_{10}	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30
x_{11}	No	No	No	No	None	\$	No	No	Thai	0-10
x_{12}	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60

Goal
WillWait
 $y_1 = \text{Yes}$
 $y_2 = \text{No}$
 $y_3 = \text{Yes}$
 $y_4 = \text{Yes}$
 $y_5 = \text{No}$
 $y_6 = \text{Yes}$
 $y_7 = \text{No}$
 $y_8 = \text{Yes}$
 $y_9 = \text{No}$
 $y_{10} = \text{No}$
 $y_{11} = \text{No}$
 $y_{12} = \text{Yes}$



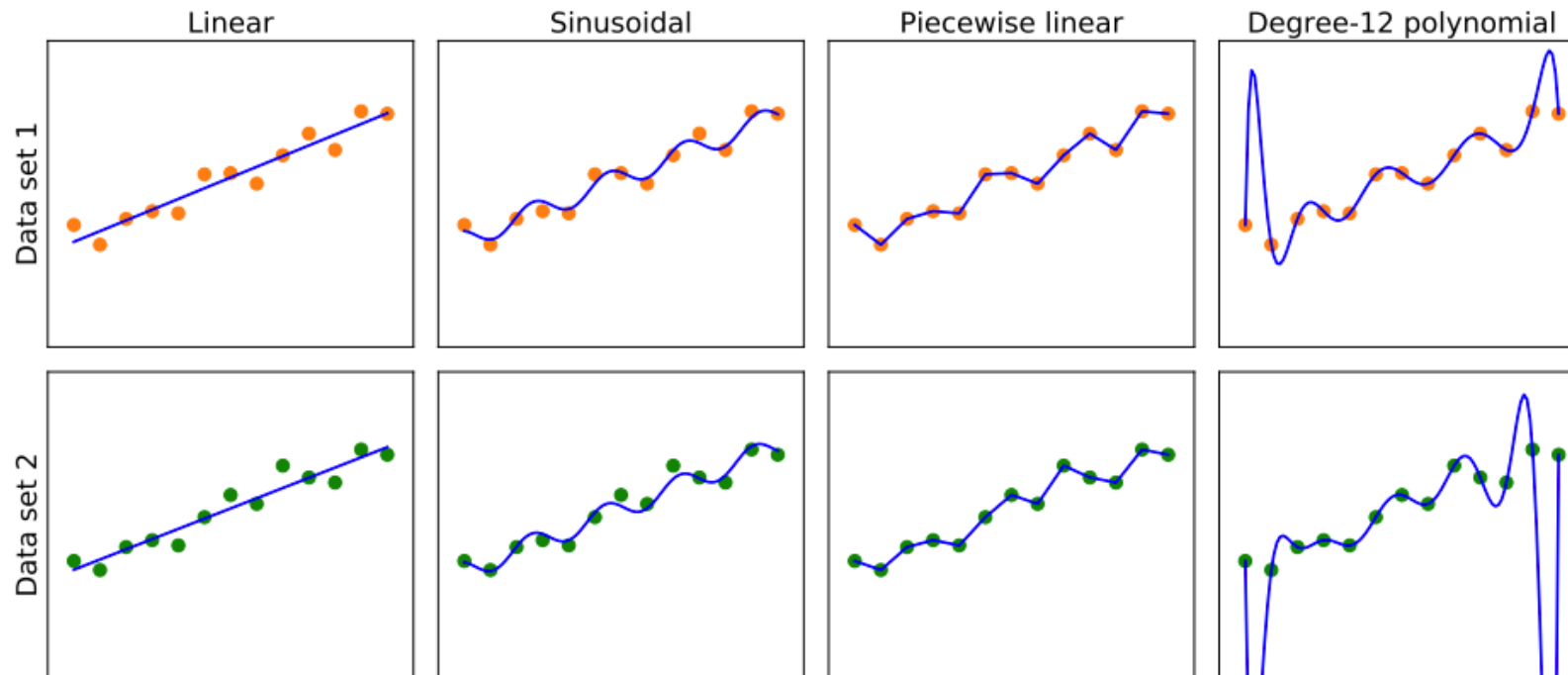
Ensemble Approaches

Overview

- Create a collection (**ensemble**) of hypotheses (**base models**)
- Combine their predictions by averaging, voting, or another level of machine learning
- Combined model: **ensemble model**

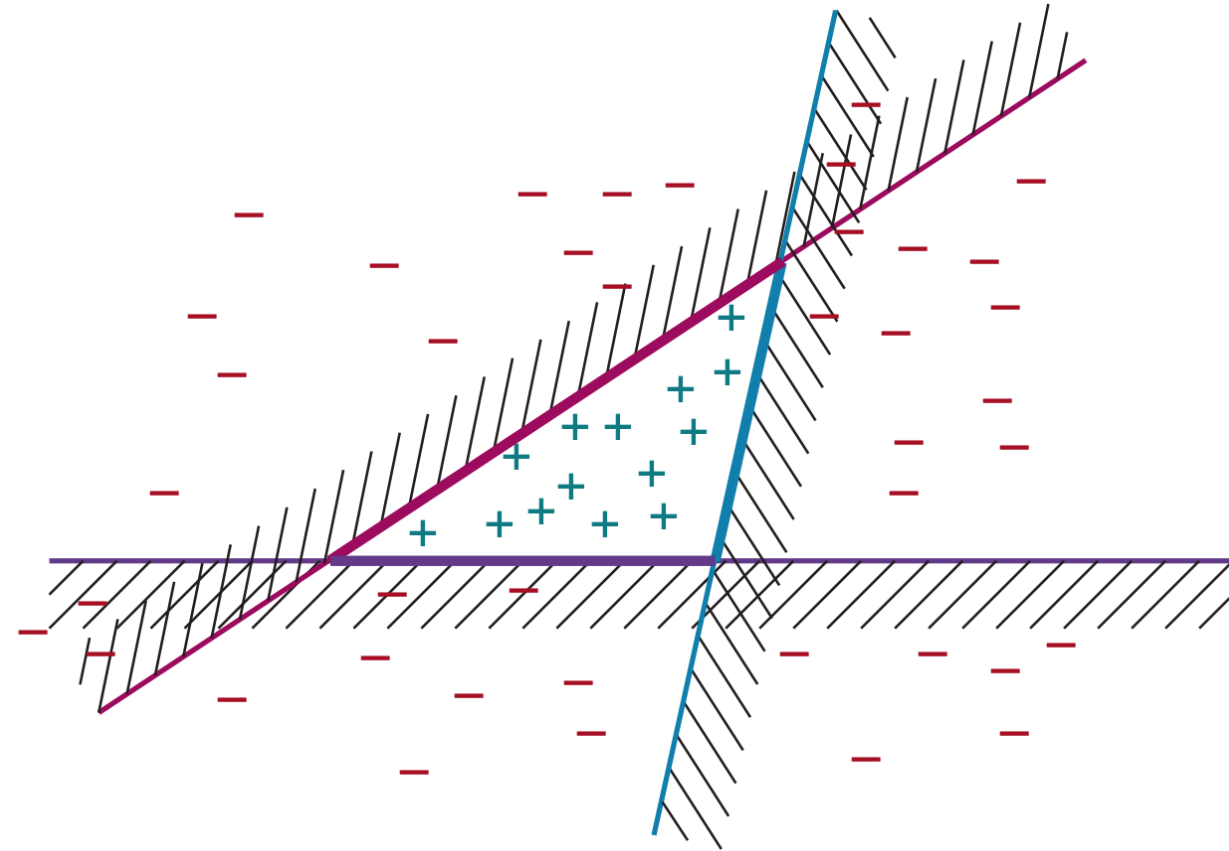
Remember Bias-Variance Trade-off?

- Review: Which models have low bias? High bias? Low variance? High variance?



Why use an ensemble approach?

- Benefit 1: Reduce bias (base models may be too restrictive)
- Example: three linear classifiers can represent a triangular region (one linear classifier can't do this!)

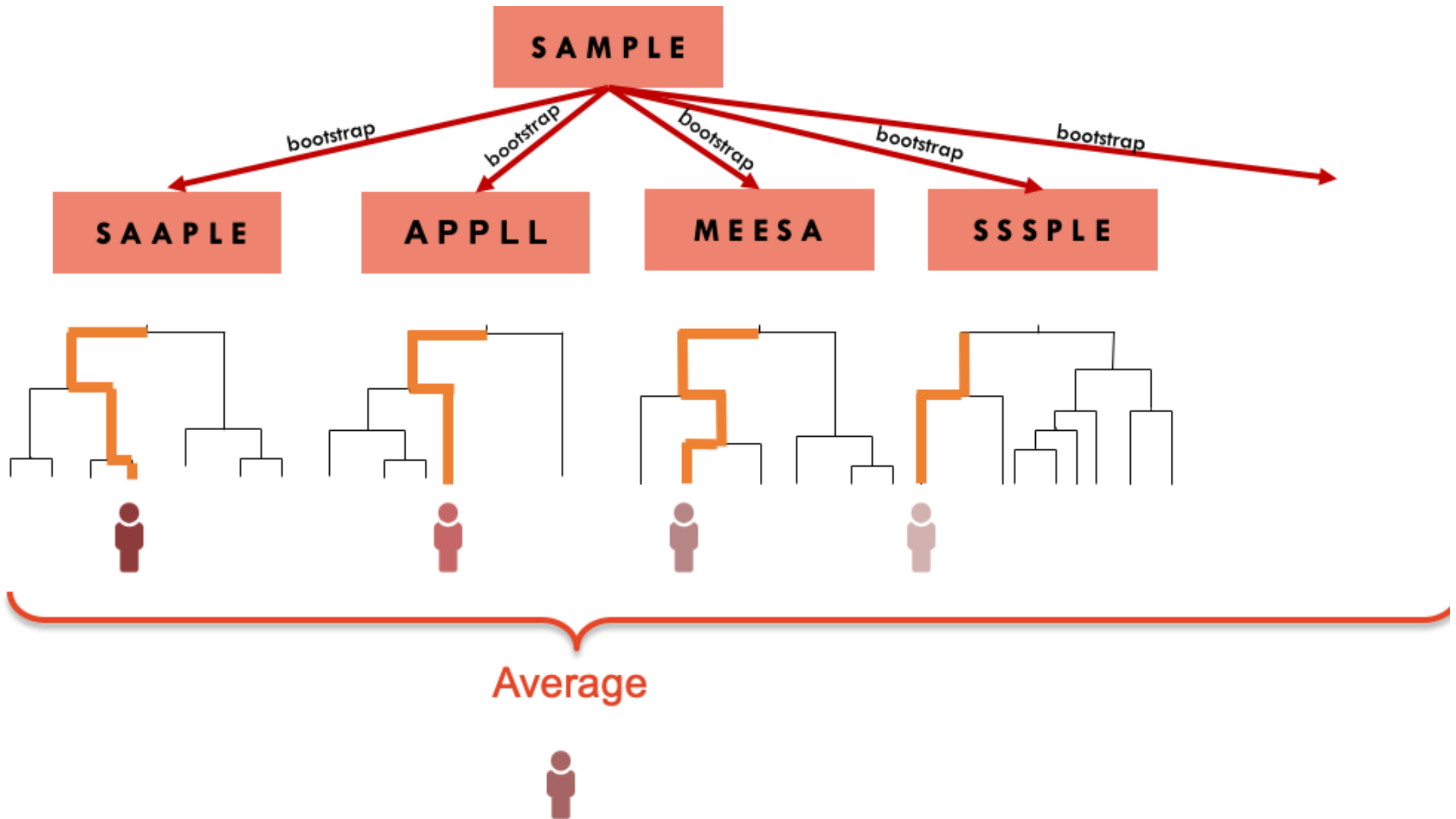


Why use an ensemble approach?

- Benefit 2: Reduce variance
- Let's say we have 5 binary classifiers, combined using majority voting
 - In order to be incorrect, 3 out of 5 classifiers must be wrong
 - If each classifier is correct 75% of the time, the ensemble will be correct 89% of the time!
 - If you have 17 classifiers correct 75% of the time, the ensemble will be correct 99% of the time!

Bagging

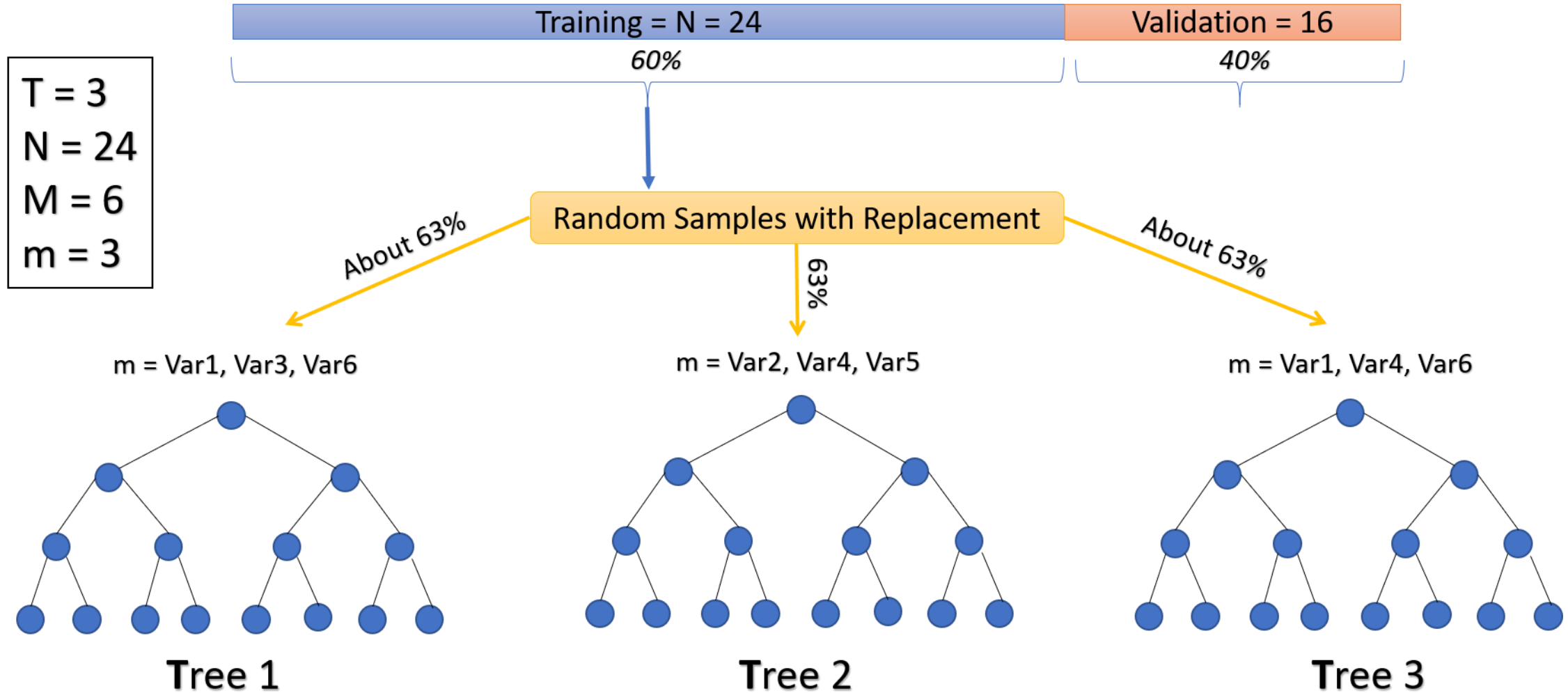
- Steps:
 - Generate K distinct training sets by sampling with replacement from the original training set
 - Train classifiers on each distinct training set
- Make a decision:
 - Classification: majority vote
 - Regression: take the average -
$$h(x) = \frac{1}{K} \sum_{i=1}^K h_i(x)$$
- Benefit: tends to reduce variance, used commonly in decision trees



Random Forests (RF)

- Motivation:
 - Bagging decision trees produces very similar trees!
 - Take steps to introduce diverse trees
- Random forest:
 - Key idea: randomly vary *attribute choices* rather than training examples
 - At each split point, select a random sampling of attributes, use the attribute with the highest information gain
 - Add bagging as well
- Benefit: Complex, unpruned models

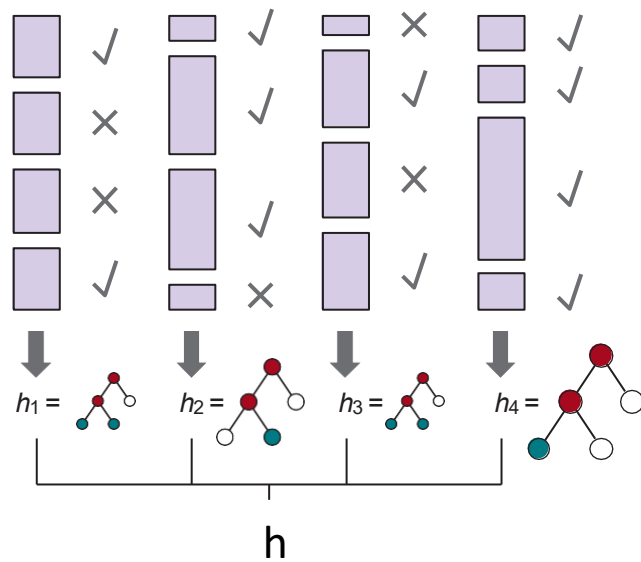
Random Forest Simplified



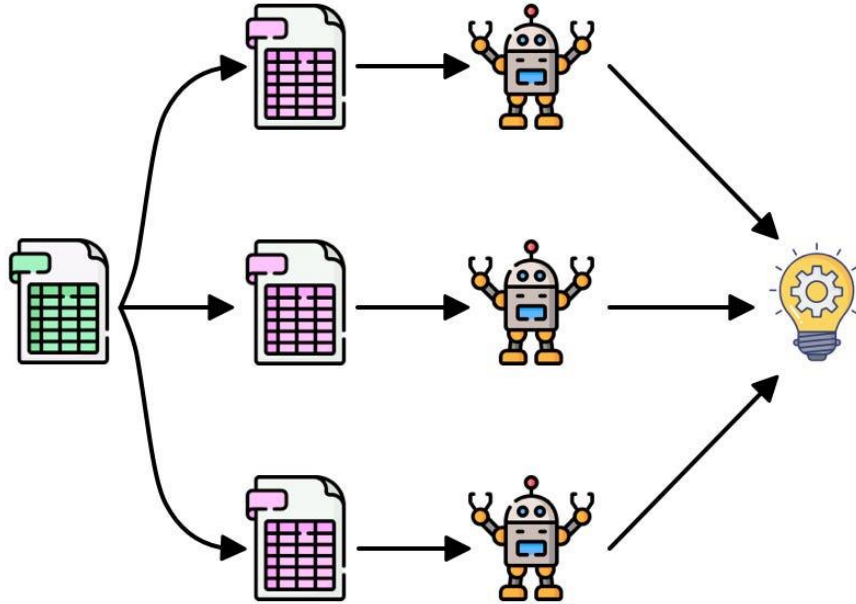
Boosting

- Key component: a weighted training set (each sample has a weight, $w_j \geq 0$) – think of it as having multiple copies of the same example
- Steps:
 1. Start with equal weights, generate first hypothesis (h_1)
 2. Increase weights that h_1 misclassified, decrease weights of correctly classified examples
 3. Then, generate next hypothesis (h_2)
 4. Continue until K hypotheses have been generated
 5. Then, vote over all hypotheses: $h(\mathbf{x}) = \sum_{i=1}^K z_i h_i(\mathbf{x})$ (z_i are new weights)
 1. Hypotheses that did better on their respective weighted training sets have more weight

Boosting, graphically

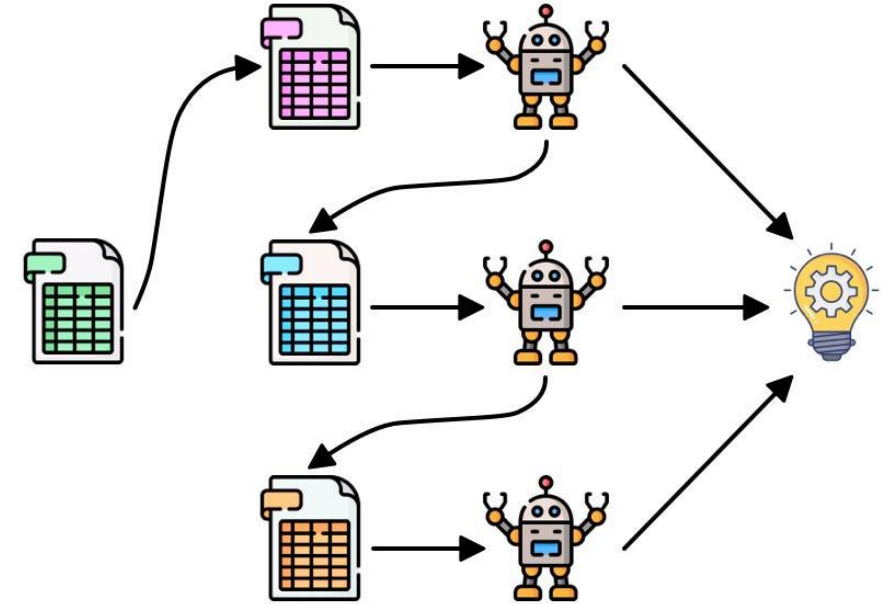


Bagging

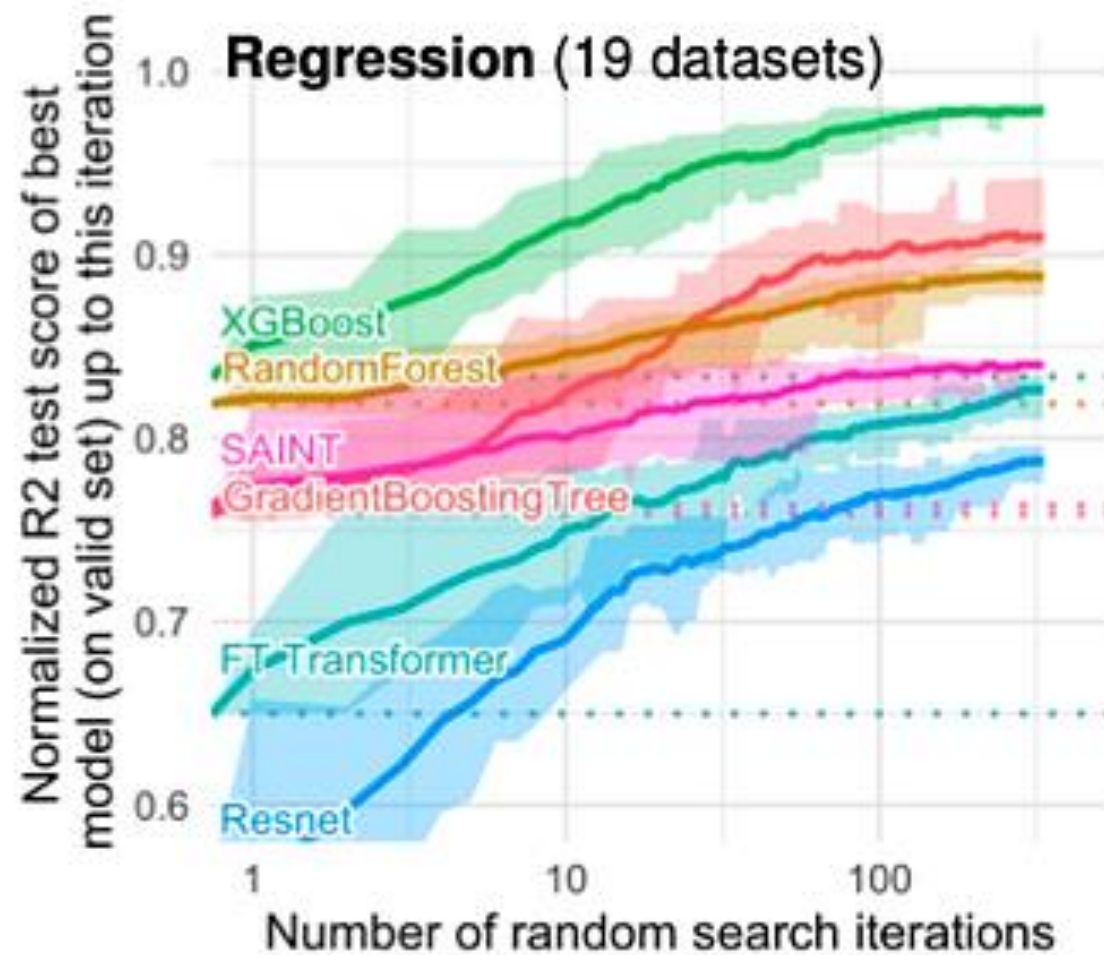
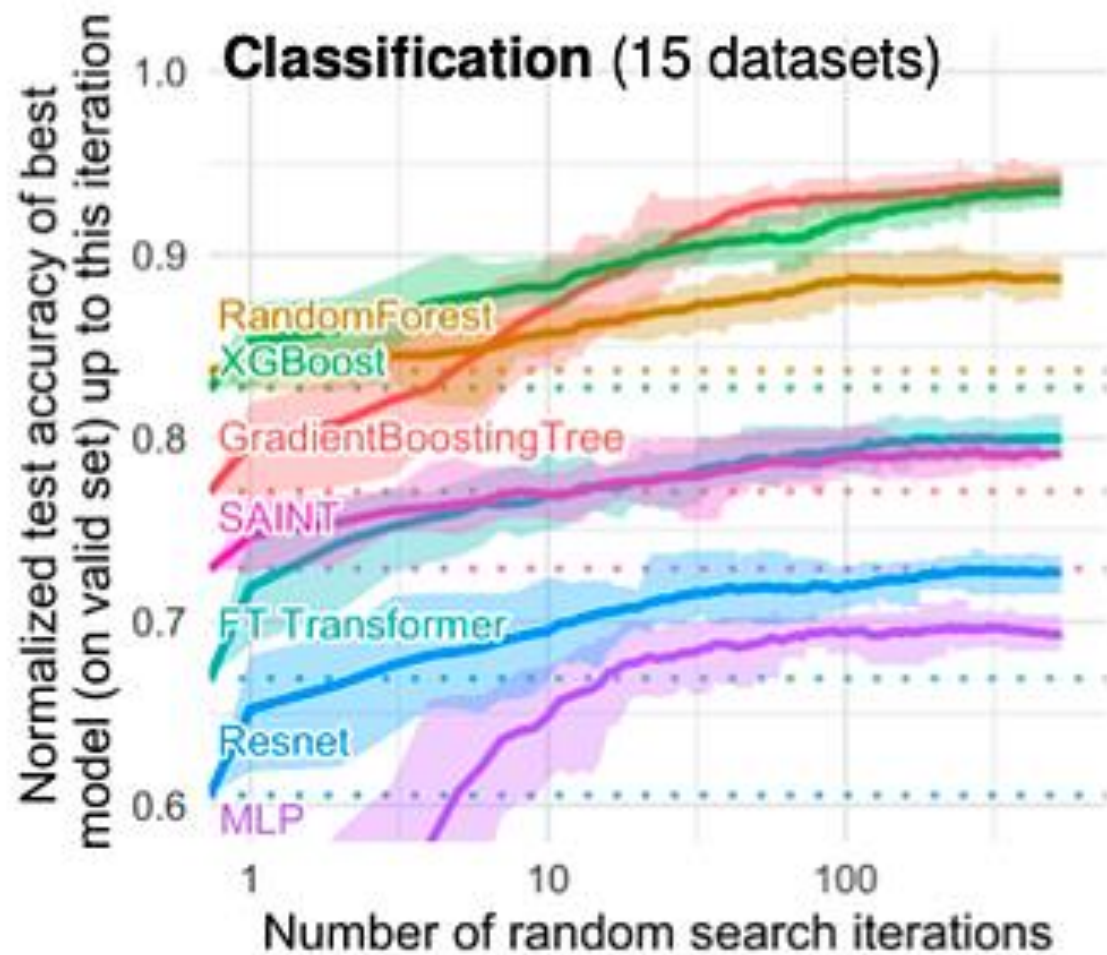


Parallel

Boosting



Sequential



Grinsztajn and Oyallon. Why do tree-based models still outperform deep learning on tabular data? NeurIPS 2022



Why are decision trees and variants some of the most preferred methods in machine learning?
Write three keywords.



I thought deep learning was
all you need...

Linear regression
Logistic regression
Decision trees
Random forests
...

scikit-learn

Machine Learning in Python

[Getting Started](#)[Release Highlights for 1.4](#)[GitHub](#)

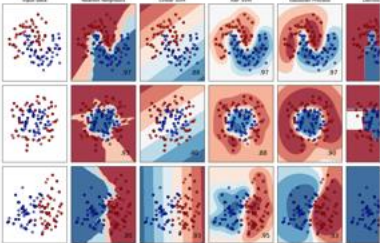
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: Gradient boosting, nearest neighbors, random forest, logistic regression, and more...

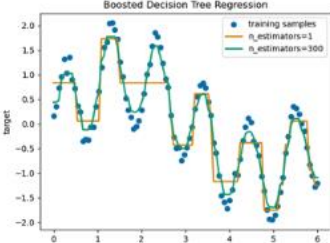


Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: Gradient boosting, nearest neighbors, random forest, ridge, and more...



Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, HDBSCAN, hierarchical clustering, and more...

